

# Étape 1

## Collecte des données

Récupérer l'historique et le temps réel de Binance, et les ramener à un format unique

|           |                                                          |
|-----------|----------------------------------------------------------|
| Projet    | CryptoBot, bot de trading piloté par le machine learning |
| Formation | Data Engineer, DataScientest                             |
| Dépôt     | DataScientest-Studio/jul26_bde_crypto                    |
| Date      | 30 septembre 2026                                        |

# Sommaire

---

## 1. Objectif de l'étape

## 2. Notions de base

2.1 La bougie

2.2 La paire

2.3 API REST et WebSocket

2.4 Les limites d'utilisation

## 3. Choix de périmètre

3.1 Trois profils de trading

3.2 Début de l'historique

## 4. Mise en œuvre

4.1 Pagination de l'historique

4.2 Respect du quota

4.3 Une fonction générique

4.4 Le flux temps réel

## 5. Nettoyage et schéma commun

5.1 Ce que renvoie Binance

5.2 Le schéma pivot

5.3 Stockage des fichiers

## 6. Contrôle qualité

6.1 Résultat

6.2 La complétude dépend de l'échelle

## 7. Suites données

## 8. Annexe : fichiers

## 1. Objectif de l'étape

La première étape consiste à identifier une source de données de marché, à comprendre ce qu'elle renvoie et à écrire une fonction de récupération générique, capable de collecter n'importe quelle paire sur n'importe quel pas de temps. Le résultat attendu est un jeu de données propre, documenté, et un échantillon qui en montre la forme.

| Demande                                         | Réalisation                                         | Emplacement                                                          |
|-------------------------------------------------|-----------------------------------------------------|----------------------------------------------------------------------|
| Choisir une source et en comprendre les données | API publique de Binance, REST et WebSocket          | <code>src/binance_rest.py</code> ,<br><code>src/binance_ws.py</code> |
| Fonction de récupération générique              | paramétrée par paire, pas de temps et date de début | <code>scripts/collect_history.py</code>                              |
| Nettoyage                                       | conversion des types, dates en UTC, schéma commun   | <code>src/preprocessing.py</code>                                    |
| Exemple de données                              | extrait commenté                                    | <code>samples/exemple_donnees_binance.json</code>                    |
| Contrôle qualité                                | doublons, valeurs nulles, cohérence, complétude     | <code>docs/rapport_qualite.json</code>                               |

## 2. Notions de base

### 2.1 La bougie

Un marché produit des milliers de transactions par minute. Pour les étudier, on les regroupe par tranche de temps. Une **bougie** (en anglais *candlestick*, souvent abrégé *kline* dans les API) résume une tranche avec cinq nombres, appelés OHLCV :

| Champ  | Signification                                 |
|--------|-----------------------------------------------|
| Open   | prix de la première transaction de la tranche |
| High   | prix le plus haut atteint                     |
| Low    | prix le plus bas atteint                      |
| Close  | prix de la dernière transaction               |
| Volume | quantité échangée pendant la tranche          |

La durée de la tranche est le **pas de temps** : une minute, quinze minutes, une heure, un jour. Une bougie de 4 heures contient exactement l'information de quatre bougies d'une heure, résumée.

Binance ajoute des champs utiles au-delà de l'OHLCV : le volume exprimé en dollars (*quote volume*), le nombre de transactions et la part du volume initiée par des acheteurs (*taker buy volume*). Ce dernier champ mesure qui a pris l'initiative de l'échange : un acheteur pressé qui accepte le prix affiché, ou un vendeur pressé.

## 2.2 La paire

Sur une plateforme d'échange, on n'achète pas « du bitcoin » dans l'absolu, on échange une monnaie contre une autre. **BTCUSDT** désigne le bitcoin coté en USDT. L'USDT est une monnaie numérique dont la valeur suit le dollar américain ; Binance l'utilise à la place du dollar, qui n'y est pas coté directement.

Nous avons retenu cinq paires parmi les plus échangées : BTC, ETH, BNB, SOL et XRP, toutes contre USDT. Un marché très actif produit des bougies régulières, sans trous ni prix aberrants dus à l'absence d'échanges.

## 2.3 API REST et WebSocket

Binance propose deux façons d'accéder aux données, qui répondent à deux questions différentes.

|                      | API REST                                                | WebSocket                                        |
|----------------------|---------------------------------------------------------|--------------------------------------------------|
| Principe             | le client pose une question, le serveur répond une fois | le client s'abonne, le serveur envoie en continu |
| Question             | que s'est-il passé ?                                    | que se passe-t-il maintenant ?                   |
| Usage dans le projet | constituer l'historique                                 | suivre le marché en direct                       |

Les deux sont publiques pour les données de marché : aucune inscription ni clé n'est nécessaire.

## 2.4 Les limites d'utilisation

Une API publique protège ses serveurs par des limites. Binance en impose deux qui ont structuré le code :

1. **1 000 bougies au maximum par appel.** Une demande plus large est tronquée sans erreur. Pour récupérer six ans de bougies, il faut enchaîner des centaines d'appels.
2. **Un budget de 6 000 unités de « poids » par minute et par adresse IP.** Chaque appel coûte un poids ; dépasser le budget renvoie une erreur 429, et insister après une 429 conduit à un bannissement temporaire (erreur 418).

# 3. Choix de périmètre

## 3.1 Trois profils de trading

Un bot ne regarde pas le marché à la même échelle selon sa stratégie. Nous avons défini trois profils, chacun avec ses pas de temps et sa profondeur d'historique.

| Profil      | Pas de temps | Historique            | Justification de la profondeur                                                        |
|-------------|--------------|-----------------------|---------------------------------------------------------------------------------------|
| Scalping    | 1m, 5m, 15m  | 180 jours             | au-delà de quelques mois, la façon dont le marché s'échange à la minute a trop changé |
| Day trading | 15m, 1h, 4h  | 730 jours             | deux ans couvrent plusieurs régimes de marché (hausse, baisse, stagnation)            |
| Swing       | 4h, 1d, 1w   | depuis septembre 2020 | une bougie hebdomadaire ne produit qu'une cinquantaine de lignes par an               |

Deux pas de temps sont partagés : le 15m (scalping et day trading) et le 4h (day trading et swing). Nous retenons pour chacun la profondeur la plus longue : collecter large satisfait les deux profils, l'inverse laisserait l'un d'eux sans données. On obtient ainsi sept collectes, une par pas de temps distinct, et non neuf.

Le profil day trading est devenu le profil principal à l'étape 3 : c'est sur lui que porte le modèle final.

### 3.2 Début de l'historique

L'historique commun commence en septembre 2020, parce que la paire SOLUSDT n'existe sur Binance que depuis août 2020. BTC et ETH remontent à 2017, mais les comparer aux autres paires sur des périodes différentes fausserait toute analyse croisée.

## 4. Mise en œuvre

### 4.1 Pagination de l'historique

La récupération d'un historique long enchaîne les appels. Le principe est simple, mais un détail change tout : on repart à chaque fois de **l'horodatage de la dernière bougie reçue**, et non d'un compteur.

```

curseur = date de début
tant que curseur < maintenant :
    lot = GET /api/v3/klines?symbol=...&interval=...&startTime=curseur&limit=1000
    si lot est vide : fin
    ajouter lot
    curseur = ouverture de la dernière bougie du lot + 1 ms

```

Avancer d'un compteur (« j'ai demandé 1 000 bougies, j'avance de 1 000 intervalles ») suppose que chaque lot est complet. Si Binance renvoie moins de bougies, parce que la plateforme était arrêtée par exemple, le compteur saute des données sans le signaler. Avancer sur la dernière bougie reçue rend la pagination juste quel que soit le contenu des lots.

## 4.2 Respect du quota

Chaque réponse de Binance indique le poids déjà consommé dans la minute (en-tête `x-mbx-used-weight-1m`). Le client le lit, se met en pause avant d'approcher la limite, et respecte le délai `Retry-After` en cas d'erreur 429. Le coût d'un appel est le même pour 1 bougie ou 1 000 : on demande donc toujours le maximum.

Si le serveur principal (`api.binance.com`) ne répond pas, le client bascule sur le miroir public `data-api.binance.vision`, qui sert les mêmes données de marché avec le même format.

## 4.3 Une fonction générique

Ni la paire, ni le pas de temps, ni la date ne sont écrits en dur. Ils sont des paramètres, et les listes de paires et de profils vivent dans un seul fichier de configuration (`src/config.py`). Collecter une nouvelle paire ne demande aucune modification de code :

```
python -m scripts.collect_history --profile day_trading
python -m scripts.collect_history --profile all
python -m scripts.collect_history --pairs ADAUSD --intervals 4h --start 2024-01-01
```

## 4.4 Le flux temps réel

Le WebSocket envoie la bougie en cours à chaque transaction qui la modifie, soit une quinzaine de messages par minute pour une bougie d'une minute. Une seule de ces versions est définitive : celle qui porte l'indicateur `x = true`, envoyée à la clôture. Le collecteur ne garde que celle-ci. Sur une mesure de 200 secondes, il a reçu 500 messages et conservé 20 bougies : sans ce filtre, 96 % des lignes enregistrées seraient des doublons à des prix jamais confirmés.

Une connexion WebSocket peut se couper. Le collecteur se reconnecte automatiquement, en espaçant ses tentatives de plus en plus (1, 2, 4, 8 secondes...) pour ne pas saturer le serveur pendant une panne.

# 5. Nettoyage et schéma commun

## 5.1 Ce que renvoie Binance

Voici une bougie telle que l'API REST la renvoie :

```
[1787918400000, "79604.12000000", "79727.36000000", "79306.11000000",
"79421.05000000", "400.05317000", 1787921999999, "31801379.67835110",
113842, "172.31828000", "13699285.03758190", "0"]
```

Quatre problèmes empêchent de l'utiliser telle quelle.

1. **Les valeurs n'ont pas de nom.** Leur sens dépend de leur position, fixée par la documentation de Binance. La liste des noms est déclarée une seule fois dans le code et associée aux positions.

2. **Les prix sont des chaînes de caractères.** Binance les envoie entre guillemets pour ne pas perdre de précision en JSON. Tant qu'ils restent du texte, les comparaisons sont fausses : `"9000"` est considéré comme plus grand que `"79600"`, comme dans un dictionnaire. Ils sont convertis en nombres.
3. **Les dates sont des nombres de millisecondes depuis 1970.** Elles sont converties en dates, en précisant explicitement le fuseau UTC. Sans fuseau, la bibliothèque supposerait l'heure locale et décalerait tout l'historique de deux heures.
4. **La réponse ne contient ni la paire ni le pas de temps.** L'API suppose que le client se souvient de sa question. Ces deux colonnes sont ajoutées, sans quoi il serait impossible de réunir plusieurs séries dans une même table.

## 5.2 Le schéma pivot

REST et WebSocket décrivent le même objet, une bougie, sous deux formes différentes : un tableau positionnel d'un côté, un objet à clés d'une lettre de l'autre (`o`, `h`, `l`, `c`...). Les deux sont convertis vers un **schéma pivot** unique de 13 colonnes :

| Colonne                                                                       | Type     | Rôle                            |
|-------------------------------------------------------------------------------|----------|---------------------------------|
| <code>symbol</code> , <code>interval</code>                                   | texte    | paire et pas de temps           |
| <code>open_time</code> , <code>close_time</code>                              | date UTC | bornes de la bougie             |
| <code>open</code> , <code>high</code> , <code>low</code> , <code>close</code> | réel     | prix OHLC                       |
| <code>volume</code> , <code>quote_volume</code>                               | réel     | volume en actif et en dollars   |
| <code>nb_trades</code>                                                        | entier   | nombre de transactions          |
| <code>taker_buy_base</code> , <code>taker_buy_quote</code>                    | réel     | volume initié par les acheteurs |

La suite du projet ne connaît que ce schéma et ignore d'où vient la donnée. C'est lui qui a servi de modèle aux tables de l'étape 2.

## 5.3 Stockage des fichiers

Deux dossiers correspondent à deux usages :

- `data/raw/` garde les réponses brutes de Binance, en JSON, telles qu'elles ont été reçues. Si le nettoyage évolue, on le rejoue depuis ces fichiers sans redemander six ans de données à Binance.
- `data/processed/` garde les bougies nettoyées, en Parquet, un format en colonnes compressé et typé, lisible directement par pandas.

## 6. Contrôle qualité

Chaque jeu de données passe quatre contrôles :

| Contrôle           | Méthode                                                                       |
|--------------------|-------------------------------------------------------------------------------|
| Doublons           | unicité de la clé (paire, pas de temps, ouverture)                            |
| Valeurs nulles     | aucune colonne vide après conversion                                          |
| Cohérence des prix | le plus haut est supérieur ou égal à l'ouverture, à la clôture et au plus bas |
| Complétude         | nombre de bougies présentes, divisé par le nombre attendu sur la période      |

La clé d'unicité inclut le pas de temps. Dans une première version, elle ne comprenait que la paire et l'heure d'ouverture : une bougie d'une heure et une bougie de quatre heures ouvertes à la même heure se seraient écrasées l'une l'autre dès que plusieurs pas de temps étaient collectés.

## 6.1 Résultat

| Pas de temps | Depuis         | Jeux      | Lignes           | Complétude |
|--------------|----------------|-----------|------------------|------------|
| 1m           | mars 2026      | 5         | 1 299 275        | 100 %      |
| 5m           | mars 2026      | 5         | 259 860          | 100 %      |
| 15m          | août 2024      | 5         | 350 620          | 100 %      |
| 1h           | août 2024      | 5         | 87 655           | 100 %      |
| 4h           | septembre 2020 | 5         | 65 655           | 100 %      |
| 1d           | septembre 2020 | 5         | 10 945           | 100 %      |
| 1w           | septembre 2020 | 5         | 1 560            | 100 %      |
| <b>Total</b> |                | <b>35</b> | <b>2 075 570</b> |            |

La collecte complète prend un peu plus de dix minutes et occupe 132 Mo en Parquet. Aucun doublon, aucune valeur nulle, aucune incohérence de prix.

## 6.2 La complétude dépend de l'échelle

Binance a connu une dizaine d'arrêts de quelques heures entre 2020 et 2023. En pas horaire sur cette période, ils apparaissent comme des bougies manquantes (99,96 % de complétude). En pas de 4 heures, ils disparaissent : lors de l'arrêt du 25 avril 2021, la bougie de 4 h qui couvre la panne existe bien, mais avec 224 transactions contre plus de 100 000 d'habitude. Le trou est absorbé dans une bougie au volume anormalement faible.

Deux règles en découlent :

- un taux de complétude n'a de sens qu'accompagné du pas de temps sur lequel il est mesuré ;
- les bougies manquantes ne doivent pas être comblées par des prix inventés. Le marché était fermé ; remplir ces heures créerait une information qui n'a jamais existé.

## 7. Suites données

Cette étape a laissé une question ouverte, sur quelle durée prédire, qui a été tranchée à l'étape 3 : le modèle final prédit le sens de la prochaine bougie sur le profil day trading. La collecte manuelle décrite ici a ensuite été automatisée à l'étape 5 : un traitement Airflow récupère les bougies manquantes toutes les 15 minutes, en repartant de la dernière bougie en base.

## 8. Annexe : fichiers

| Fichier                                           | Rôle                                                         |
|---------------------------------------------------|--------------------------------------------------------------|
| <code>src/config.py</code>                        | paires, profils, adresses de Binance, limites                |
| <code>src/binance_rest.py</code>                  | client REST : pagination, quota, bascule sur le miroir       |
| <code>src/binance_ws.py</code>                    | client WebSocket : filtre des bougies clôturées, reconnexion |
| <code>src/preprocessing.py</code>                 | schéma pivot, conversions, contrôle qualité                  |
| <code>scripts/collect_history.py</code>           | collecte de l'historique par profil                          |
| <code>scripts/collect_stream.py</code>            | écoute du flux temps réel                                    |
| <code>samples/exemple_donnees_binance.json</code> | exemple de données brutes et nettoyées                       |